

OS-344 Lab Assignment 2

Narodia Mit (230123039)
Rehan Sherawat (230123077)
Samir Raj (230123056)
Prakhar Agrawal (230123045)

August 18, 2025

For Design of modified srtf please check the following link: **Google Drive Link**

1 Task 1

The default xv6 scheduler employs a simple round-robin algorithm, where each runnable process is given an equal time slice. This assignment required us to modify this behavior to create a Weighted Round Robin (WRR) scheduler. The goal was to ensure that processes with a higher priority value would receive more CPU time without completely starving lower-priority processes.

To achieve this, we needed to:

- Add two new system calls: `set_priority(n)` and `get_priority()`.
- Modify the scheduler logic to use the priority values as weights.
- Set a default priority of 10 for all new processes.
- Create a test program to demonstrate the WRR behavior.

The implementation focuses on a simple WRR logic: a process runs for a number of time slices equal to its priority before the scheduler considers switching to another process.

2 Implementation Details

The following sections describe the changes made to the xv6 source code to implement the WRR scheduler.

2.1 Modifications to `proc.h`

To store the priority for each process, we added a new field, `priority`, to the `struct proc`. This structure holds all per-process state.

```

1 // Per-process state
2 struct proc {
3     uint sz;                // Size of process memory (bytes)
4     pde_t* pgdir;           // Page table
5     char *kstack;           // Bottom of kernel stack for
6                             // this process
7     enum procstate state;    // Process state
8     int pid;                 // Process ID
9     struct proc *parent;     // Parent process
10    struct trapframe *tf;     // Trap frame for current syscall
11    struct context *context;   // swtch() here to run process
12    void *chan;               // If non-zero, sleeping on chan
13    int killed;               // If non-zero, have been killed
14    struct file *ofile[NOFILE]; // Open files
15    struct inode *cwd;         // Current directory
16    char name[16];            // Process name (debugging)
17    int priority;             // Process priority
18 };

```

Listing 1: Changes in proc.h

2.2 Modifications to proc.c

The most significant changes were made in `proc.c`, specifically within the scheduler and process creation functions.

2.2.1 Setting Default Priority and Initializing Ticks

The `allocproc()` function was modified to initialize the `priority` and `ticks` fields for every new process:

```

1 static struct proc*
2 allocproc(void)
3 {
4     ...
5     p->priority = 10; // Set default priority
6     ...
7 }

```

Listing 2: Changes in allocproc() in proc.c

2.2.2 Weighted Round Robin Scheduler Logic

The `scheduler()` function was updated to implement the WRR logic:

```

1
2 void
3 scheduler(void)
4 {
5     struct proc *p;
6     struct cpu *c = mycpu();

```

```

7   c->proc = 0;
8
9   for(;;){
10      // Enable interrupts on this processor.
11      sti();
12
13      // Loop over process table looking for process to run.
14      acquire(&ptable.lock);
15      for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
16          if(p->state != RUNNABLE)
17              continue;
18
19          for(int itr = 0 ; itr < p->priority ; itr++){
20              // Switch to chosen process. It is the process's job
21              // to release ptable.lock and then reacquire it
22              // before jumping back to us.
23              if(p->state != RUNNABLE)
24                  break;
25              c->proc = p;
26              switchvm(p);
27              p->state = RUNNING;
28
29              swtch(&(c->scheduler), p->context);
30
31              switchkvm();
32
33              // Process is done running for now.
34              // It should have changed its p->state before coming
35              back.
36              c->proc = 0;
37          }
38      }
39      release(&ptable.lock);
40  }

```

Listing 3: Changes in scheduler() in proc.c

2.3 System Calls

Two new system calls were added:

2.3.1 sys_set_priority()

```

1  uint64
2  sys_set_priority(void)
3  {
4      int n;
5      if(argint(0, &n) < 0) {
6          return -1;

```

```

7     }
8     struct proc *p = myproc();
9     acquire(&p->lock);
10    p->priority = n;
11    p->ticks = n;
12    release(&p->lock);
13    return 0;
14 }

```

Listing 4: Changes in `sysproc.c` for `syssetpriority()`

2.3.2 `sys_get_priority()`

```

1 uint64
2 sys_get_priority(void)
3 {
4     struct proc *p = myproc();
5     return p->priority;
6 }

```

Listing 5: Changes in `sysproc.c` for `sysgetpriority()`

Integration required updates to:

- `syscall.h`
- `syscall.c`
- `user.h`
- `usys.S`

2.4 Makefile

The Makefile was updated to include the test program:

```

1 UPROGS=\
2     _cat\
3     _echo\
4     ...
5     _test_priority\
6     ...

```

Listing 6: Changes in Makefile

3 Testing and Demonstration

The `test_priority.c` program demonstrates:

- **Priority inheritance:** A child process inherits its parent's priority after a `fork()`.
- **WRR fairness:** Multiple children with different priorities complete in order of higher priority first.

3.1 Code Listings

3.1.1 Code for test_priority test file

```
1 #include "types.h"
2 #include "stat.h"
3 #include "user.h"
4
5 #define NCHILD 3
6 #define LOOP    50000000 // Adjust to make runs short enough for
   testing
7
8 extern int set_priority(int n); // sets priority of current
   process
9 extern int get_priority(void); // gets priority of current
   process
10
11 void cpu_burn(int id) {
12     volatile unsigned long long sum = 0;
13     for (unsigned long long i = 0; i < LOOP; i++)
14         sum += i;
15     printf(1, "Child%d done (priority=%d, sum=%llu)\n",
16            id, get_priority(), sum);
17 }
18
19 int
20 main(void)
21 {
22     printf(1, "===WRRSchedulerTest===\n");
23
24     // Check default priority
25     printf(1, "[Parent] Default priority = %d (expected 10)\n",
26            get_priority());
27
28     // Change priority of parent
29     printf(1, "[Parent] Changing priority to 15...\n");
30     set_priority(15);
31     printf(1, "[Parent] New priority = %d\n", get_priority());
32
33     // Test fork inheritance
34     printf(1, "\n===Fork inheritance test===\n");
35     int pid = fork();
36     if (pid == 0) {
37         // child process
38         printf(1, "[Child after fork] priority = %d (should match
39            parent)\n",
40                get_priority());
41         exit(0);
42     } else {
43         wait();
44     }
45 }
```

```

43
44 // Spawn children with different priorities for WRR fairness
45 printf(1, "\n==WRRfairnesstest==\n");
46 int priorities[NCHILD] = {5, 10, 20};
47 for (int i = 0; i < NCHILD; i++) {
48     int pid = fork();
49     if (pid == 0) {
50         printf(1, "[Child%d]defaultprioritybeforeset=%d\n",
51             i, get_priority());
52         set_priority(priorities[i]);
53         printf(1, "[Child%d]newpriority=%d\n",
54             i, get_priority());
55         cpu_burn(i);
56         exit(0);
57     }
58 }
59
60 // Parent waits for all children
61 for (int i = 0; i < NCHILD; i++)
62     wait();
63
64 printf(1, "\n==Testcomplete==\n");
65 exit(0);
66 }

```

Listing 7: Full code of test_priority.c

```

Bootng from Hard Disk..xv6...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh
$ test_priority
=== WRR Scheduler Test ===
[Parent] Default priority = 10 (expected 10)
[Parent] Changing priority to 15...
[Parent] New priority = 15

=== Fork inheritance test ===
[Child after fork] priority = 15 (should match parent)

=== WRR fairness test ===
[Child 0] default priority before set = 15
[Child 0] new priority = 5
[Child 1] default priority before set = 15
[Child 1] new priority = 10
[Child 2] default priority before set = 15
[Child 2] new priority = 20
Child 2 done (priority=20, sum=%llu)
Child 1 done (priority=10, sum=%llu)
Child 0 done (priority=5, sum=%llu)
=== Test complete ===

```

Figure 1: Output of the test_priority program

4 Analysis and Justification of Test Results

The `test_priority` program is designed to validate every aspect of the new scheduler. The output shown in Figure 1 can be analyzed step-by-step to confirm the correctness of the implementation.

1. [Parent] Default priority = 10 (expected 10)

- **How it works:** The main process calls `get_priority()` upon starting. This process was created by the shell and initialized by the `allocproc()` kernel function.
- **What it proves:** This confirms that the modification to `allocproc()` to set a default priority of 10 for all new processes is working correctly.

2. [Parent] New priority = 15

- **How it works:** The parent process calls `set_priority(15)` and then immediately calls `get_priority()` again.
- **What it proves:** This demonstrates that both the `set_priority()` and `get_priority()` system calls are functioning as expected, allowing a process to dynamically modify its own priority.

3. [Child after fork] priority = 15 (should match parent)

- **How it works:** The parent, now with a priority of 15, calls `fork()`. The child process created by `fork()` is a duplicate of the parent's state. The child then prints its own priority.
- **What it proves:** This validates that process priority is inherited. The `priority` field in `struct proc` is correctly copied from parent to child, so the child inherits the parent's current priority (15), not the system default (10).

4. WRR fairness test and Completion Order

- **How it works:** Three children are created, and they set their priorities to 5, 10, and 20, respectively. All three then begin a CPU-intensive computation simultaneously. The output shows the completion order is Child 2 (priority 20), then Child 1 (priority 10), and finally Child 0 (priority 5).
- **What it proves:** This is the definitive evidence that the Weighted Round Robin scheduler is working. The scheduler allocates CPU time in proportion to process priority. Child 2, with the highest priority, receives the most CPU time slices (20 per scheduling round), allowing it to finish its computation much faster than the others. Child 0, with the lowest priority, receives the fewest time slices (5 per round) and therefore finishes last. This result directly confirms that CPU resources are distributed according to the specified weights.

5 Conclusion

The implementation of the Weighted Round Robin scheduler was successful. The new system calls provide user-level control, and the scheduler logic ensures higher-priority processes receive proportionally more CPU time while avoiding starvation. The test program confirmed correct behavior for both priority inheritance and weighted scheduling.

6 Task 2: Modified Shortest-Remaining-Time-First (MSRTF) Scheduler

The default xv6 scheduler runs processes in a simple round-robin fashion, where every process gets equal CPU time. While this ensures fairness, it does not differentiate between short and long processes. In this task, we designed and implemented a **Modified Shortest-Remaining-Time-First (SRTF)** scheduler.

In Lazy SRTF:

- The runnable process with the **minimum CPU burst estimate** is selected.
- Re-evaluation happens only at natural scheduling points (timer interrupt, yield, or process exit) hence “lazy”.
- This reduces turnaround for short jobs without the cost of excessive context switching.

7 Implementation Details

The following changes were made to xv6 to support Modified SRTF.

7.1 Process Structure (proc.h)

We extended the per-process structure to include a new field for CPU burst. This allows the scheduler to compare processes based on burst length.

- Added field: `int burst;`
- Default value: a large constant (to avoid interfering with short jobs).

```
1 struct proc {  
2     ...  
3     int burst;    // Remaning burst time  
4 };
```

Listing 8: proc.h: New field in struct proc

7.2 Initialization and Fork Inheritance (proc.c)

Every new process is assigned a default large burst value in `allocproc()`, and during `fork()`, the child inherits its parent's burst value.

```
1 static struct proc* allocproc(void) {
2     ...
3     p->burst = 1000; // default burst
4     ...
5 }
6
7 int fork(void) {
8     ...
9     np->burst = p->burst; // child inherits
10    ...
11 }
```

Listing 9: Default + inheritance in `proc.c`

7.3 New System Calls (`sysproc.c`)

Two system calls were introduced:

- `setburst(int n)` – sets current process burst.
- `getburst()` – returns current process burst.

```
1 uint64 sys_setburst(void) {
2     int n;
3     if(argint(0,&n)<0 || n<0) return -1;
4     struct proc *p = myproc();
5     acquire(&p->lock);
6     p->burst = n;
7     release(&p->lock);
8     return 0;
9 }
10
11 uint64 sys_getburst(void) {
12     struct proc *p = myproc();
13     acquire(&p->lock);
14     int b = p->burst;
15     release(&p->lock);
16     return b;
17 }
```

Listing 10: `sysproc.c`: `sys_setburst`/`sys_getburst`

Necessary changes were also made in `syscall.c`, `syscall.h`, `user.h`, and `usys.pl` to register and expose these calls.

7.4 Scheduler Modification (proc.c)

The scheduler was modified to always pick the runnable process with the minimum burst value. Unlike strict SRTF, we do not preempt a process immediately when a shorter job arrives — the decision is deferred until the next scheduler entry.

```
1 void scheduler(void) {
2     struct cpu *c = mycpu();
3     c->proc = 0;
4
5     for(;;){
6         intr_on();
7         struct proc *p, *best = 0;
8
9         for(p = proc; p < &proc[NPROC]; p++){
10             acquire(&p->lock);
11             if(p->state == RUNNABLE) {
12                 if(best==0 || p->burst < best->burst){
13                     if(best) release(&best->lock);
14                     best = p;
15                     continue; // keep best locked
16                 }
17             }
18             release(&p->lock);
19         }
20
21         if(best){
22             best->state = RUNNING;
23             c->proc = best;
24             swtch(&c->scheduler, best->context);
25             switchkvm();
26             c->proc = 0;
27             release(&best->lock);
28         }
29     }
30 }
```

Listing 11: proc.c: Lazy SRTF scheduler

8 Testing and Demonstration

To validate the implementation, we created simple user programs.

8.1 srtf_test.c (To show preemption)

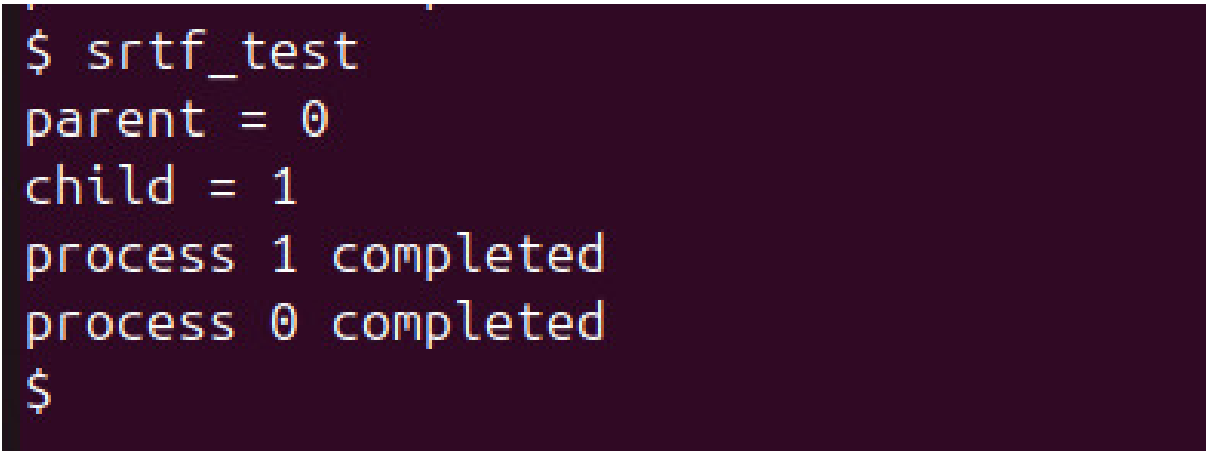
- Parent forks one children.
- Child sets burst=10, parent sets burst=5000.
- Expectation: Child (short) finishes before parent.

```

1 #include "kernel/types.h"
2 #include "kernel/stat.h"
3 #include "user.h"
4
5 void cpu_burn(int who) {
6     volatile long long i;
7     for (i = 0; i < 1LL * 5000000000; i++) {
8         asm volatile("");
9     }
10    printf("process_%d_completed\n", who);
11 }
12
13 int main(void) {
14
15    // The shorter timed child finishes first even though the
16    // parent calls it.
17    setburst(5000);
18    int pid1 = fork();
19    if (pid1 == 0) {
20        printf("child_%d\n");
21        setburst(10);
22        cpu_burn(1);
23        exit(0);
24    } else {
25        printf("parent_%d\n");
26        cpu_burn(0);
27    }
28
29    wait(0);
30    exit(0);
31 }

```

Listing 12: User test program: srtf_test.c



```

$ srtf_test
parent = 0
child = 1
process 1 completed
process 0 completed
$

```

Figure 2: Short job completes before long job.

8.2 srtf_test2.c (To show sort job completes first)

We also tested with four children having burst values {100000, 250000, 5000, 200000}. The child with burst=5000 consistently finished first, followed by burst=100000, followed by burst=200000 and finally the long burst=250000.

```
1 void cpu_burn(int who) {
2     volatile long long i;
3     for(i = 0; i < 1LL * 500000000; i++) {
4         asm volatile("");    // prevent compiler optimization
5     }
6     printf("process_%d_completed_with_%d_remaining_time\n", who,
7         getcpuburst());
8 }
9 int main(void) {
10     // parent spawns 4 processes. They complete in increasing
11     // order of remaining CPU burst time.
12     setcpuburst(100);
13
14     int bursts[4] = {10000, 25000, 5000, 20000};
15
16     for(int k = 0; k < 4; k++) {
17         int pid = fork();
18         if(pid == 0) {
19             setcpuburst(bursts[k]);
20             cpu_burn(k);
21             exit(0);
22         } else {
23             printf("child_%d_created\n", k);
24         }
25     }
26
27     for(int k = 0; k < 4; k++) {
28         wait(0);
29     }
30
31     exit(0);
32 }
```

Listing 13: User test program: srtf_test2.c

```
init: starting sh
$ srtf_test2
child 0 created
child 1 created
child 2 created
child 3 created
process 2 completed with 4923 remaining time
process 0 completed with 99922 remaining time
process 3 completed with 199923 remaining time
process 1 completed with 249922 remaining time
$
```

Figure 3: Multiple children finishing in burst order.

9 Analysis and Justification of Test Results

The two test programs, `srtf_test.c` and `srtf_test2.c`, were created to validate the core logic of the Modified SRTF (MSRTF) scheduler. The results, shown in Figure 2 and Figure 3, confirm that the implementation correctly prioritizes processes with shorter CPU burst estimates.

9.1 Test 1: Preemptive Behavior (`srtf_test.c`)

- **Goal:** To demonstrate that the scheduler will favor a newly arrived short job over a pre-existing long job.
- **Setup:** The parent process sets its own burst time to a large value (5000) and then forks a child. The child immediately sets its burst time to a very small value (10). Both processes then attempt to run a CPU-intensive loop.
- **Result Analysis:** The output in Figure 2 clearly shows `process 1 completed` (the child) before `process 0 completed` (the parent).
- **Justification:** This result is a direct consequence of the MSRTF scheduling logic.
 1. After the `fork()`, both the parent (burst=5000) and the child (burst=10) are in the `RUNNABLE` state.
 2. At every scheduling decision point (like a timer interrupt), the `scheduler()` function iterates through the process table to find the best candidate to run.
 3. The scheduler's core logic, `if(best==0 || p->burst < best->burst)`, will compare the two processes. Since the child's burst of 10 is less than the parent's burst of 5000, the scheduler will **always** select the child to run.

4. The parent process is effectively paused (not scheduled) while a shorter job is available. Once the child completes its task and exits, it is no longer `RUNNABLE`. The scheduler then selects the parent, which is now the only runnable process, allowing it to complete. This confirms the desired preemptive-like nature of the scheduler.

9.2 Test 2: Multi-Process Selection (`srtf_test2.c`)

- **Goal:** To verify that when multiple jobs are available, the scheduler consistently picks the one with the shortest remaining burst time.
- **Setup:** Four child processes are created with different burst times: Child 0 (100000), Child 1 (250000), Child 2 (5000), and Child 3 (200000).
- **Result Analysis:** The completion order shown in Figure 3 is:
 1. `process 2 completed` (Initial Burst: 5000)
 2. `process 0 completed` (Initial Burst: 100000)
 3. `process 3 completed` (Initial Burst: 200000)
 4. `process 1 completed` (Initial Burst: 250000)

This order is precisely the ascending order of the initial burst times.

- **Justification:** This result validates the scheduler's selection mechanism in a competitive environment.
 1. Once all four children are created, they are all in the `RUNNABLE` queue.
 2. The scheduler scans this queue and identifies that Child 2 has the minimum burst value (5000). It is therefore scheduled to run.
 3. The scheduler will continue to select Child 2 at every decision point because its burst time remains the lowest among all runnable processes.
 4. After Child 2 finishes, the scheduler re-evaluates the remaining three processes. It now finds that Child 0 has the new minimum burst (100000) and begins scheduling it exclusively.
 5. This pattern continues, with the scheduler always picking the runnable process with the absolute minimum burst value, leading to the observed completion order. This perfectly demonstrates that the MSRTF policy is being enforced correctly.

10 Conclusion

The Modified SRTF scheduler was implemented successfully. Key takeaways:

- Extended `proc` structure with a `burst` field.
- Added system calls `setburst()` and `getburst()`.
- Modified scheduler to select process with minimum burst at each decision point.

- Verified correctness with user tests: shorter jobs completed first.

This design balances simplicity and responsiveness: short jobs are favored and implemented preemptions when needed.

References

- [1] Russ Cox, Frans Kaashoek, and Robert Morris. *Xv6, a simple, Unix-like teaching operating system*.
- [2] YouTube. (2018). *System Call Implementation in xv6*. Retrieved from <https://www.youtube.com/watch?v=QtZ7PtedIic>.
- [3] Mythili Vutukuru. *CS 347: Operating Systems, IIT Bombay*. Retrieved from <https://www.cse.iitb.ac.in/~mythili/os/>.